Proyecto 2: Análisis Sintáctico

Jueves 12 de Noviembre

I. Descripción

En este proyecto, Ud deberá desarrollar un *parser* para el Lenguaje C. Toda la programación debe realizarse en C sobre Linux, usando las herramientas `flex` y `bison`. No se pueden cambiar las especificaciones de este documento.

Su programa recibe como entrada un programa fuente, aparentemente escrito en lenguaje C, y produce como salida un reporte de **todos** los errores sintácticos detectados. La salida se dirige a consola.

II. Ejecución

El programa se invocará desde la línea de comando de una consola. Recibirá como argumento el nombre del archivo fuente a ser procesado[1], junto con cualquier opción necesaria para ejecutar el programa al estilo tradicional de `UNIX/Linux`. Si el usuario no da los argumentos, o si estos contienen errores (en cantidad o forma) se desplegará una ayuda explicando las opciones disponibles.

III. Preproceso

En el proyecto anterior se desarrolló un preprocesador que podía manejar las directivas `#include` y `#define` sin parámetros. Dicha funcionalidad debe estar correcta para este proyecto.

IV. Errores Sintácticos

Nótese que el preproceso y el componente léxico serán básicamente los mismos del proyecto programado previo (corregirlos si tuvieran problemas pendientes).

Con la ayuda de `bison`, Ud. escribirá un *parser* del lenguaje C **completo**. Debe detectar la mayor cantidad posible de errores sintácticos y generar **buenos mensajes de error**, señalando el lugar dentro del fuente original donde se da el error sintáctico de la manera más precisa posible. Por otro lado, si su programa reporte errores que no son sintácticos ni léxicos, el proyecto será calificado con un 0.

V. Salida

Por defecto, los errores de sintaxis encontrados se reportan a consola (al estilo de los compiladores de Linux como `gcc`). Se debe deplegar la línea del fuente donde ocurre el error y un indicador llamativo de la posición más aproximada posible al lugar donde se detectó el error. Si no se encuentran errores de sintaxis no se da ningún mensaje adicional.

[1] este archivo puede tener cualquier extensión

VI. Pruebas

El día de la demostración cada grupo debe contar con al menos 8 programas en C, 4 de los cuales deben ser **grandes**[2]. De sus 8 programas, 4 no deben mostrar ningún error de sintaxis, y 4 de ellos deben mostrar múltiples errores sintácticos.

VII. Reglas y Requisitos Indispensables

El irrespeto a una sola de las siguientes reglas o la ausencia de uno solo de los siguientes requisitos vuelve al proyecto "no revisable" y recibe un 0 de calificación inmediata:

- La colaboración entre grupos se considera fraude académico.
- Utilizar código, archivos de datos, pruebas o documentación de semestres previos (*e.g.*, buscando en `GitHub`) o usar IA para completar cualquier parte de su trabajo no sólo es fraude académico, es una aceptación de incompetencia e inutilidad.
- Todo el código debe estar escrito en C (no C++).
- Deben usar `flex`.
- El proyecto debe compilar y ejecutar en Linux. Todo debe estar **integrado**, explicaciones del tipo "*todo está bien pero no pudimos pegarlo*"[3] provocan la cancelación automática de la revisión.
- La presentación debe ser de mucha calidad.
- Para el momento de la revisión, su programa no debe tener "prints" de depuración. Luce muy poco profesional.
- La única interacción válida con el programa es por medio de argumentos de línea de comando.
- No debe dar "Segmentation Fault" bajo ninguna circunstancia.
- La demostración debe hacerse en una máquina que levante Linux de manera real (puede ser dual), es decir no usar máquinas virtuales. No se puede levantar de un disco o memoria externa. No se pueden prestar máquinas entre grupos.

VIII. Fecha de Entrega

Demostraciones el **Jueves 12 de Noviembre** a las 6:00pm en el aula usual de clases. Mande además un `.tgz` con todo lo necesario (fuentes, makefile, readme, etc.) a `torresrojas.cursos.02@gmail.com`. Ponga como subject: `[COM-UCR] Proyecto 2 - Fulano - Mengano - etc.`, donde Fulano, Mengano y etc. son los miembros del grupo.

[2] al menos 500 líneas de código

[3] esto incluye los supuestos casos cuando alguien del grupo de trabajo no hizo su parte – el profesor no está interesado en sus problemas de organización.